PUBLIC WHITEPAPER

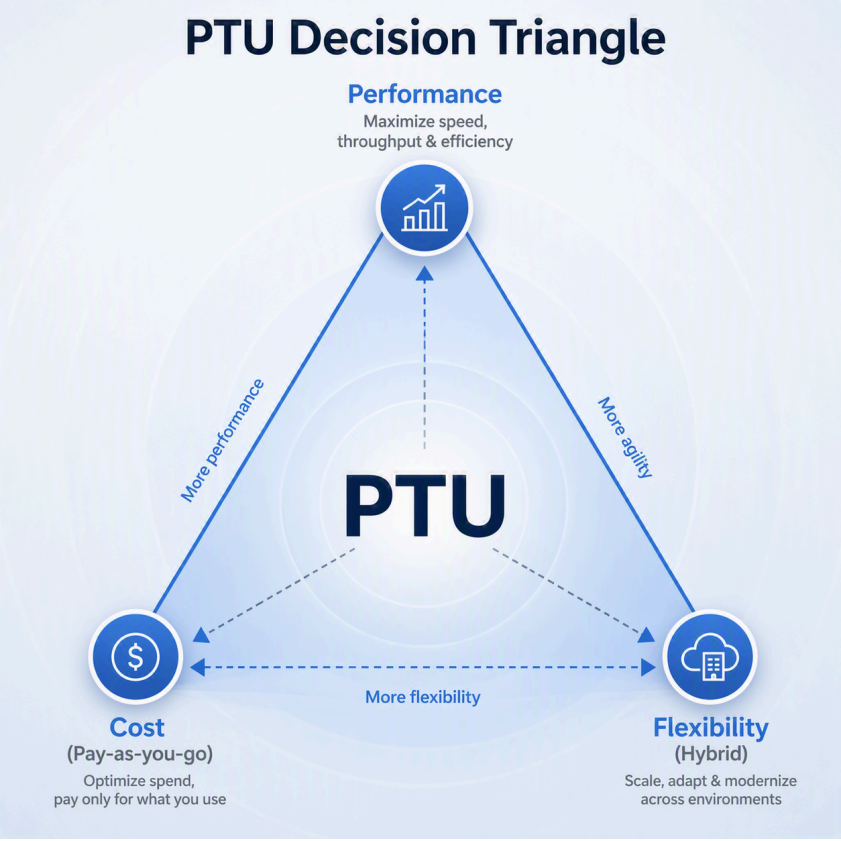
Architecture-Driven Capacity Planning for Enterprise AI Workloads

Moving beyond sizing toward intelligent deployment decisions

Linda Zhang · Microsoft · June 2026

PUBLIC-FACING NOTE

This paper is intended as technical guidance for capacity planning discussions. It is not a replacement for official Microsoft documentation, the Microsoft Foundry capacity calculator, Azure pricing tools, or customer-specific benchmarking.



Core architecture model — a reserved baseline layer (PTU) handles predictable demand; an elastic burst layer (PAYGO / Standard) absorbs spikes.

Executive Summary

As enterprises scale generative AI into production, capacity planning often becomes one of the most important architecture decisions. Teams commonly ask, “How much capacity do I need?” The better question is, “What deployment model should support this workload reliably, cost-effectively, and at production scale?”

This whitepaper introduces a decision-first approach to AI workload capacity planning. The goal is to help teams align deployment models with workload behavior, balance performance and cost, and transition from experimentation to production with greater confidence.

- Align capacity strategy to traffic shape, token usage, latency sensitivity, and operational requirements.
- Separate steady-state baseline demand from burst or peak demand.
- Use real workload signals and benchmarking to validate assumptions before committing to production capacity.
- Treat sizing as a validation step after the architecture pattern has been selected.

Why AI Deployments Stall

In many production AI programs, capacity planning happens too late. Architecture decisions are made first, and sizing is treated as a downstream calculation. That sequence creates avoidable risk: teams can over-commit to dedicated capacity, under-plan for latency-sensitive usage, or miss the economics of hybrid deployment patterns.

Common planning gaps

- **Decision happens too late:** capacity model selection is delayed until after the workload architecture is already mostly fixed.
- **Static inputs dominate:** teams use average request volumes without understanding peak behavior, output-token mix, cache impact, or traffic variability.
- **Architecture awareness is missing:** a calculator may help estimate capacity, but it does not always answer whether dedicated capacity, usage-based capacity, or a hybrid model is the right starting point.
- **Telemetry is underused:** many planning exercises rely on assumptions instead of observed token usage and production traffic patterns.

> **POINT OF VIEW**
>
> Capacity planning is not just a math problem. It is an architecture decision, and it should be framed that way from the beginning.

A Decision-First Planning Model

A decision-first model starts with workload behavior, then maps that behavior to the right architecture pattern. Only after that should teams size capacity, validate cost, and prepare deployment operations.

Step 1 — Understand workload behavior

- Is traffic predictable, spiky, or seasonal?
- Are requests latency-sensitive or batch-oriented?
- What are the average and peak requests per minute?
- What is the input/output token mix?
- How much of the workload can benefit from prompt caching?
- Are there data residency, governance, or isolation requirements?

Step 2 — Select the architecture pattern

Architecture pattern	Best fit	Planning implication
Dedicated / baseline-focused	Stable, predictable, latency-sensitive production workloads	Use provisioned capacity for steady-state demand; validate with official sizing tools and benchmark results.
Elastic / usage-based	Variable or bursty workloads where demand is difficult to predict	Pay-per-token or usage-based patterns can reduce over-commitment risk during early or unpredictable demand.
Hybrid baseline + elastic	Common enterprise pattern with stable baseline plus burst periods	Use dedicated capacity for baseline and elastic routing for peaks; monitor usage and optimize continuously.

Step 3 — Size and validate

Sizing should validate the selected pattern, not drive the entire architecture by itself. At this stage, teams should use official Microsoft sizing guidance and the Microsoft Foundry capacity calculator, benchmark representative traffic, and validate cost with the current pricing model.

Design Principle: Baseline vs. Peak

The core planning principle is simple: **design for baseline, scale for peak.** Baseline demand is the predictable portion of the workload that can justify dedicated capacity. Peak demand is the burst portion that should be treated separately rather than forcing the entire system to be sized to rare spikes.

- **Baseline demand:** predictable traffic that can be served with committed or dedicated capacity.
- **Peak demand:** less predictable traffic that may require spillover, queueing, throttling strategy, elastic capacity, or usage-based handling.
- **Optimization loop:** monitor actual usage, compare against assumptions, and revisit the architecture as the workload matures.

Bringing Data Into the Decision

Modern AI systems generate usage signals that should be part of the capacity planning process. Planning based only on estimates creates avoidable error. Planning with telemetry creates a repeatable decision model that can improve over time.

- Requests per minute and traffic distribution over time
- Input tokens, output tokens, and prompt-cache behavior
- P95 / P99 traffic patterns, not only average values
- Latency and utilization metrics from test or production deployments
- Deployment type, quota, availability, and regional considerations

> **PRACTICAL RECOMMENDATION**
>
> Before production rollout, benchmark with representative traffic. For existing workloads, use observed token usage and throughput patterns to seed sizing assumptions and validate the architecture pattern.

From Experimentation to Production

The move from proof of concept to production is rarely just a model-selection exercise. It requires a production platform view: deployment strategy, capacity model, monitoring, routing, cost governance, and operational ownership.

- Predictable performance and latency for user-facing or agentic applications
- Cost controls that scale with actual usage and business value
- Governance and workload isolation for multi-team enterprise deployments
- Monitoring and review cycles to prevent capacity plans from becoming stale
- A clear escalation path when quota, capacity, or deployment availability creates delivery risk

Positioning in the Enterprise AI Stack

Capacity planning should sit inside the broader enterprise AI platform layer. It connects model selection, deployment strategy, routing, API management, observability, and cost governance. When those decisions are handled together, teams can move faster without treating every workload as a one-off capacity exercise.

Platform layer	Capacity planning role
Model selection	Choose models with the right latency, cost, and throughput characteristics.
Routing and API management	Apply routing or spillover patterns based on demand shape and resiliency needs.
Observability	Use telemetry to compare real usage against planning assumptions.
Cost governance	Map capacity decisions to chargeback, reservations, utilization, and optimization reviews.
Production operations	Scale, monitor, and optimize as traffic and model portfolios evolve.

Business Impact

An architecture-first approach helps teams reduce deployment friction and improve the quality of capacity conversations across engineering, platform, and business stakeholders.

Engineering teams	Business stakeholders	Platform teams
Faster movement from design to production; fewer re-architecture cycles; clearer performance expectations	Better cost-performance alignment; reduced deployment risk; more transparent capacity rationale	Standardized patterns; improved governance and reuse; capacity planning tied to operating signals

Strategic Takeaways

- Capacity planning is an architecture decision, not only a calculation.
- Real workload behavior should guide whether a workload uses dedicated capacity, usage-based capacity, or a hybrid pattern.
- Baseline and peak demand should be modeled separately to avoid over-provisioning and production surprises.
- Telemetry and benchmarking should be used to validate assumptions before and after production rollout.
- Hybrid patterns are likely to be important for many enterprise AI workloads because they balance predictable baseline demand with elasticity for spikes.

Conclusion

As enterprise AI adoption accelerates, successful teams will differentiate not only by what they build, but by how reliably and economically they operate AI workloads at scale. A decision-first, architecture-aware planning model shifts the conversation upstream: from “how many PTUs do we need?” to “what operating model best fits this workload?”

That shift helps teams make better deployment decisions, reduce capacity planning friction, and create sustainable production systems that can evolve as usage patterns change.

> *Most calculators tell you how much capacity to estimate. An architecture-driven approach helps you decide whether dedicated capacity is the right model in the first place — and what deployment*

> *pattern to build around it.*

References and Further Reading

- Microsoft Learn — Determine PTU sizing for a workload: learn.microsoft.com/.../provisioned-throughput-sizing
- Microsoft Learn — Provisioned throughput for Foundry Models: learn.microsoft.com/.../provisioned-throughput
- Microsoft Learn — Provisioned throughput billing and cost management: learn.microsoft.com/.../provisioned-throughput-billing

---